

Multi-Link Safety Filtering for VLA Policies Around Moving Hazards

Yatharth Agarwal

School of Electrical and Computer Engineering
Purdue University, West Lafayette, IN, USA

agarw414@purdue.edu

Vijay Raghunathan

School of Electrical and Computer Engineering
Purdue University, West Lafayette, IN, USA

vr@purdue.edu

Abstract—A vision–language–action (VLA) policy can finish a manipulation task while knocking over objects unrelated to it, so task success alone does not show that the policy is safe to deploy in clutter. We study how to keep a pretrained VLA policy clear of such hazards at run time without retraining it, which requires guarding more of the arm than the end effector, following the hazard as it moves, and sharing onboard compute with the policy. Our training-free shield covers the gripper, wrist, and forearm with five ellipsoids and filters every commanded motion through one barrier program against a keep-out ellipsoid fitted from RGB-D perception at reset. Sparse optical flow then carries that ellipsoid’s center along with the hazard, with no repeated detection or refitting. Over six simulated hazard-motion conditions, the shield lowers collision from 65.62% to 27.27% and raises safe-success, task completion without collision, from 29.35% to 50.43%. Ablations show that guarding the arm links protects beyond end-effector shielding, and that tracking recovers most of the protection lost when the hazard estimate is frozen at reset. On heterogeneous edge hardware, the five-ellipsoid barrier runs on the CPU in 2.2 ms at the 99th percentile, and trimming the vision–language prefix and taking fewer flow-matching steps shortens each $\pi_{0.5}$ policy call on the integrated GPU from 343 to 177.3 ms. On a physical SO-101 arm across four tasks, the arm touched the hazard in 3 of 16 shielded episodes versus 11 of 16 unshielded ones. Project page: <https://yatharth.github.io/multilink-safety-filter/>

I. INTRODUCTION

Robot manipulation is shifting from task-specific controllers to general policies that follow language instructions: vision–language–action (VLA) policies act across diverse tasks and scenes [1]–[3], and multimodal large language models now command robot arms directly from camera images [4]. In shared workspaces, task completion alone is insufficient: a robot reaching for a dish may complete the task while its forearm knocks over a nearby glass. The commanded goal does not account for the cost of disturbing that unrelated object, and near sharp tools or people, the same contact can cause injury. With moving hazards, the $\pi_{0.5}$ [1] flow-matching policy completes 62.93% of episodes but disturbs the hazard in 65.62% of episodes. Protecting against such a hazard requires constraints that extend beyond the task objective, and those constraints must follow it as it moves.

Fine-tuning on collision-free demonstrations can encode avoidance in a policy’s weights [5], but it depends on demonstrations that represent the hazard. A runtime

This work was supported in part by CoCoSys and CogniSense, two of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

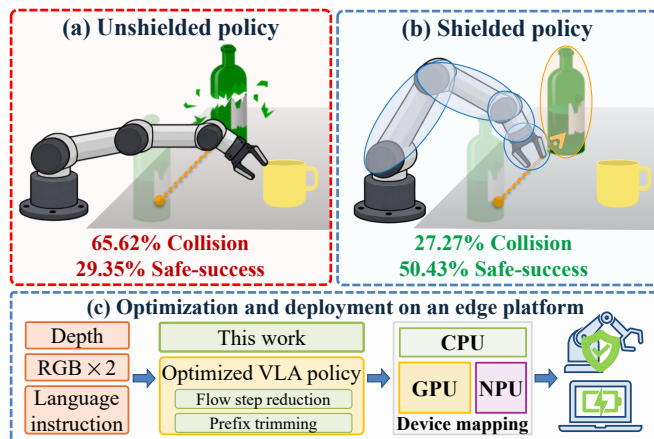


Fig. 1. **Multi-link shielding around a pretrained policy.** (a,b) The unshielded policy and our shield on the moving-hazard benchmark in simulation. (c) Policy optimization and a measured device mapping place the complete loop on one heterogeneous edge platform.

safety filter instead imposes a keep-out constraint on a hazard designated at execution time, without retraining; control-barrier filters modify the nominal command to satisfy it [6], [7], and existing VLA safety filters establish this interface [8], [9]. Because the filter acts only on commanded motion, it is compatible with diverse safety-fine-tuned policies or a multimodal language model.

Deployment, however, requires protection across the arm links that sweep the workspace, updates to the keep-out region as it moves, and execution within a compute budget shared with the policy. Onboard execution keeps control and its protection independent of network availability and latency. These requirements are coupled: broader coverage protects only if the hazard estimate is current, and keeping it current adds computation to the control loop.

We present a runtime shield that addresses these requirements alongside a pretrained policy, without changing its weights. At reset, a vision–language model (VLM) names the hazard, an open-vocabulary detector localizes it, and registered depth yields a three-dimensional ellipsoid. The barrier program guards five ellipsoids across the gripper, wrist, and forearm, and sparse optical flow transports the hazard ellipsoid’s center as the hazard moves, without re-detection or refitting. On a laptop processor with a CPU, an integrated GPU (iGPU), and a neural processing unit (NPU), the policy runs on the iGPU, the fastest device measured for it, while the millisecond-scale barrier and tracker stay on the CPU.

The evaluation compares the complete system against

the unshielded policy and runtime-filter baselines on the static-hazard SafeLIBERO benchmark [8] and a new moving-hazard benchmark. Collision captures protection, task success captures task performance, and safe-success captures both. Ablations isolate the effects of arm coverage, hazard estimation, tracking, and policy configuration, while edge measurements characterize the cost of running the shield alongside the policy.

These requirements pose a single question: can a fixed VLA policy be protected from a task-irrelevant moving object beyond its end effector, with a hazard estimate that stays current, within an onboard compute budget? The contributions answer it along three axes: coverage, motion, and deployment:

- 1) **Coverage: multi-link runtime shielding for pretrained policies.** Five guarded ellipsoids extend the end-effector barrier across the gripper, wrist, and forearm by adding one constraint per element to a single barrier program. Broader coverage reduces collisions under image-based perception without a significant task-success cost.
- 2) **Motion: hazard transport without re-detection, and a moving-hazard benchmark.** Sparse optical flow transports the hazard ellipsoid without refitting. A benchmark of six hazard-motion conditions tests protection: our shield reduces collisions from 65.62% to 27.27% and increases safe-success from 29.35% to 50.43%.
- 3) **Deployment: heterogeneous edge characterization and mapping.** The policy call is measured on the CPU, the iGPU, and the NPU, and the complete loop then runs under the latency-minimizing mapping: policy, detector, and hazard namer on the iGPU, barrier and tracker on the CPU. Every policy configuration is evaluated with the shield active, and the same mapping drives a physical SO-101 arm around a tracked moving hazard.

II. RELATED WORK

Safety filters for learned manipulation. Control barrier functions enforce forward invariance under their model assumptions by constraining a nominal controller through a quadratic program [6], [7]; robust variants incorporate bounded state-estimation error [10]. Recent systems impose related constraints on VLA policies by fitting a semantic barrier at reset [8], grounding constraints from policy attention [9], or enforcing them within the action decoder [11], [12]. However, end-effector constraints leave the wrist and forearm unguarded as they move through the scene. Manipulator barrier filters guard the links with sphere models [13], [14], and Any-Body Guard addresses robot geometry in configuration space but uses a quasi-static scene representation [15]. We instead guard five ellipsoids spanning the gripper, wrist, and forearm against one motion-updated hazard ellipsoid.

Perception and motion. A geometric safety filter is only as current as its hazard estimate. Open-vocabulary detection

with registered depth can construct that estimate from images [16], but motion can invalidate a reset-time fit. Per-step segmentation can relocate the centers of hazard ellipsoids fitted at reset [9], and promptable video segmentation maintains object identity through streaming inference [17]; both run a neural network at every update, sharing the edge platform’s compute budget with the policy. Sparse optical flow [18] instead transports the fitted ellipsoid, preserving its reset shape.

Safety training and runtime constraints. Fine-tuning on collision-free demonstrations encodes avoidance behavior in the policy weights and adds no online filtering cost [5]. Runtime shielding complements this learned behavior by accepting a newly designated hazard at execution time and imposing an explicit geometric constraint without updating the policy. The scope here is task-irrelevant hazards that should remain untouched; tasks requiring contact with the hazard call for constraints beyond a strict keep-out region.

Edge deployment. Efficient VLA systems reduce backbone size, input tokens, or action-decoding cost [19]–[21]. For $\pi_{0.5}$, which reuses an encoded vision–language prefix during iterative action generation [1], [2], work on iterative-head policies, adaptive multi-view pruning, quantization, and cross-accelerator characterization is particularly relevant [22]–[24]. Policy efficiency alone does not determine the performance of a shielded system: hazard estimation and filtering impose workloads at different cadences, and policy approximations alter the commands presented to the shield. The deployment question is therefore how to allocate shared compute while retaining protection and task performance. This work maps perception, tracking, filtering, and policy inference to heterogeneous edge hardware at their respective cadences, and the evaluation tests each policy configuration with runtime shielding active.

III. SYSTEM DESIGN

Our shield is a training-free runtime filter between a pretrained VLA policy and the robot. The shield acts only on the policy’s commanded end-effector motion, so it does not depend on the policy’s internal architecture. The evaluation uses the $\pi_{0.5}$ flow-matching policy [1], [2], whose structure Sec. III-D exploits for edge execution. Building on AEGIS [8], we retain that system’s semantic hazard naming, open-vocabulary grounding, depth-derived ellipsoid, and ellipsoid-separation control-barrier formulation. This common interface carries three extensions: protection across the arm’s distal links, transport of the fitted barrier when the hazard moves, and execution of the complete loop on heterogeneous edge hardware. The policy returns a chunk of normalized end-effector commands; each command is filtered against the measured robot state immediately before execution. Perception and control exchange only the hazard ellipsoid, allowing motion updates without changing the barrier formulation (Fig. 2).

A. The Barrier Program

Let $\mathcal{C}$ index the guarded ellipsoids, each of which contributes a keep-out constraint. The barrier program, a control-

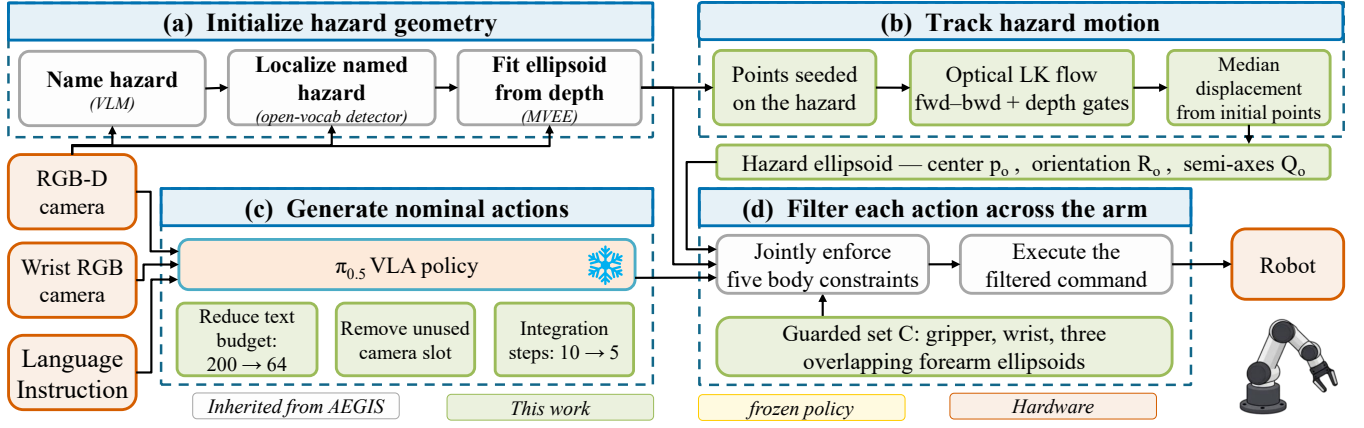


Fig. 2. **The shield runs perception and control at distinct cadences.** (a) At reset, perception names the hazard, detects it, and fits its ellipsoid. (b) Sparse Lucas–Kanade (LK) optical flow transports the ellipsoid center every five steps. (c) The policy returns 10 actions per call; the panel shows the optimized configuration. (d) Five guarded ellipsoids constrain each command at nominal 20 Hz. Perception and control exchange only the hazard ellipsoid.

barrier-function quadratic program (CBF-QP), jointly adjusts the commanded motion to satisfy these constraints while minimizing the deviation of the commanded motion from the policy’s nominal command. The hazard is represented by one ellipsoid with center $p_o \in \mathbb{R}^3$, rotation $R_o \in SO(3)$, and semi-axes $Q_o = \text{diag}(q_1, q_2, q_3)$ with $q_i > 0$,

$$\mathcal{O} = \{y \in \mathbb{R}^3 : \|Q_o^{-1}R_o^\top(y - p_o)\|_2 \leq 1\}. \quad (1)$$

For each $k \in \mathcal{C}$, the guarded ellipsoid $\mathcal{B}_k(x)$ of the form (1), with center $p_k(x)$, rotation $R_k(x)$, and semi-axes Q_k , is rigidly attached to its link; the constraint protects this modeled volume. Following rotating-hyperplane barriers [25], an auxiliary unit vector z_k selects a plane tangent to $\mathcal{B}_k(x)$, and the barrier $h_k(x, z_k)$ is the minimum signed distance from that plane to $\mathcal{O}$, positive on the side away from $\mathcal{B}_k(x)$; $h_k \geq 0$ certifies that the plane separates the two ellipsoids. The map $G_k(x) = J_k(x)J_{ee}^+(x)$, with J_k and J_{ee} the Jacobians at the center of $\mathcal{B}_k$ and at the end effector and $(\cdot)^+$ the pseudoinverse, relates the commanded end-effector twist $u \in \mathbb{R}^6$ to motion at the center of $\mathcal{B}_k$, assuming the minimum-norm joint velocity $\dot{q} = J_{ee}^+u$ realizes u . The simulated arm instead executes u through an operational-space controller, whose null-space motion admits a different $\dot{q}$, so G_k predicts rather than reproduces the link motion. At each control step, the inherited program solves

$$\begin{aligned} \min_{u, \dot{z}} \quad & \|u - u_{\text{nom}}\|_W^2 + \|\dot{z} - \dot{z}_{\text{nom}}\|_{W_z}^2 \\ \text{s.t.} \quad & a_k^u G_k(x)u + a_k^z \dot{z}_k + \alpha h_k(x, z_k) \geq 0, \quad \forall k \in \mathcal{C}. \end{aligned} \quad (2)$$

The controller executes the minimizing u^* and integrates $\dot{z}^*$ over the control step, renormalizing each z_k to unit length. Here x is the robot configuration, u_{nom} is the policy’s nominal end-effector twist, z and $\dot{z}$ stack the vectors z_k and their rates, and $\dot{z}_{\text{nom}}$ stacks the reference rates inherited from AEGIS.

The coefficients $a_k^u \in \mathbb{R}^{1 \times 6}$ and $a_k^z \in \mathbb{R}^{1 \times 3}$ are the partial derivatives of h_k with respect to the pose of $\mathcal{B}_k$ and to z_k , evaluated at the measured state. The positive-definite weighting matrices W and W_z penalize deviations from the nominal inputs, and $\alpha = 10$ sets the linear barrier gain.

B. Multi-Link Coverage

An end-effector barrier leaves the swept volume of proximal links outside its modeled safe set. The design therefore instantiates $\mathcal{C}$ with five guarded ellipsoids distributed over the gripper, wrist, and forearm. The elongated forearm link is represented by three overlapping ellipsoids that retain the link’s full cross-section and span its long axis; the remaining ellipsoids are derived directly from the links’ collision geometry. The elbow and upper arm carry no guarded ellipsoid.

Through G_k , each ellipsoid’s row in (2) constrains its own motion rather than treating it as rigidly attached to the gripper. All five rows share the same hazard ellipsoid in a single barrier program, which adds one constraint and one auxiliary vector per ellipsoid while keeping the objective, solver, and hazard representation.

C. Hazard Initialization and Tracking

Reset-time initialization The VLM names the hazard from the task instruction and external image, using a short list of candidate hazard phrases when applicable. An open-vocabulary detector grounds the phrase in each camera view, and the selected boxes crop the registered depth images. Workspace, outlier, and density filtering precede a minimum-volume enclosing ellipsoid (MVEE) fit. The selected detection fixes the object identity for the episode.

Motion updates For a static hazard, the reset ellipsoid is held fixed. For a moving hazard, the shield retains its reset orientation and semi-axes, and only its center is transported. Let p_o^0 be the fitted center, $c(t)$ a three-dimensional tracker position, and c^0 its value at the reset measurement. The update is $p_o(t) = p_o^0 + (c(t) - c^0)$, with $R_o(t) = R_o^0$ and $Q_o(t) = Q_o^0$. Using an anchored displacement rather than $c(t)$ as an absolute center estimate cancels a fixed projection offset, while retaining one hazard shape avoids frame-to-frame geometric variation. Center transport therefore assumes that the hazard translates without large rotation or change in its visible shape.

The tracker operates in the fixed external camera. At initialization, the fitted ellipsoid’s center is projected onto the image, and a regular grid of nearby pixels is retained

only where the measured depth agrees with the center depth. These persistent points are propagated by Lucas–Kanade flow [18]. Forward–backward agreement and depth consistency retain points that continue to support the hazard, and the median displacement from each point’s seed position yields a robust image-plane translation without summing incremental center updates. The camera ray passing through the translated image center intersects the horizontal plane through p_o^0 to obtain $c(t)$, assuming the hazard maintains its reset height. Two gates reject an unsupported translation: a bound on the center displacement per measurement, and a threshold on the normalized correlation with the reset appearance. When either gate fires, the center is held at its last accepted position while a bounded template search looks for the hazard; a match re-seeds the tracker.

The tracker measures this displacement every $s = 5$ control steps and holds the center between measurements, so the hazard ellipsoid updates within an action chunk.

Guarantee scope Under the standard assumptions of a fixed hazard, exact state and geometry, an initially safe state, and feasible continuous-time execution under the modeled dynamics, the barrier conditions in (2) preserve the modeled safe set $\mathcal{S} = \{(x, z) : h_k(x, z_k) \geq 0, \forall k \in \mathcal{C}\}$. The deployed system relaxes these assumptions: geometry is estimated from RGB-D observations, commands are applied in sample-and-hold fashion, link motion is approximated through G_k , and the hazard pose is updated periodically from optical flow. Consequently, each control step certifies the barrier conditions with respect to the latest estimated hazard pose rather than providing a continuous-time invariance guarantee for the moving-hazard loop. In particular, motion between updates and discrete changes in the estimated hazard pose are not represented explicitly in (2). If the program becomes infeasible, the current implementation executes and logs the nominal action rather than invoking a certified fallback. We therefore treat collision reduction in the deployed system as an empirical result. Extending the shield with perception-error bounds, sampled-data barrier conditions, and an explicit hazard-velocity term would provide a path toward stronger guarantees under moving hazards.

D. Edge-Aware Policy Execution

Each policy invocation encodes the current vision–language prefix once [1], [2], then repeatedly integrates the action expert’s flow-matching field. Attention is bidirectional within the prefix, but prefix tokens do not attend to action tokens, so the prefix keys and values are computed once per call and reused across integration steps. A new observation changes the prefix, so the cache does not carry over across calls. Each integration step attends to the cached prefix, so redundant prefix slots increase both the one-time encoding work and the repeated prefix-action attention, whereas reducing the number of integration steps acts directly on the repeated path.

Two changes trim the prefix, and a third shortens the integration loop, none of them touching the policy weights. First, the benchmark provides two camera views, while the

serving configuration allocates a third, masked camera slot. Masking does not skip that slot’s image encoder or its 256 prefix positions, so the deployed configuration removes it before encoding. Second, the padded text budget falls from 200 to 64 tokens; the longest benchmark instruction uses 21 tokens. This prefix trimming reduces the prefix from $3 \times 256 + 200 = 968$ to $2 \times 256 + 64 = 576$ positions. Third, the integration steps fall from ten to five, and the resulting approximation is evaluated with the shield active.

Workload size and cadence decide the mapping. The barrier program and the optical-flow update are millisecond-scale, issued at every nominal 50 ms control step and every fifth step, respectively. Each is shorter than the overhead of an accelerator dispatch, so both run on the CPU. Dense policy inference, issued once per ten-action chunk, requires a dedicated accelerator such as an iGPU or an NPU.

IV. BENCHMARKS AND METRICS

The evaluation covers static and moving hazards in two simulated benchmarks that share the simulator, robot, policy interface, and evaluation metrics. Image-based configurations estimate hazard geometry from rendered RGB and registered simulator depth; they do not use the exact hazard pose or shape.

Static hazards. SafeLIBERO [8] extends the LIBERO suites [26] with a designated hazard object and a collision criterion for each scene. The evaluation comprises 32 scenes: four suites $\times$ two hazard levels $\times$ four tasks. The static benchmark runs every scene, with 1,600 episodes per configuration. It compares our shield with the unshielded policy and with two runtime safety methods, AEGIS and OSCBF. Two coverage variants, end-effector-only and multi-link, share one perception pipeline and controller.

Moving hazards. A new moving-hazard benchmark reuses these scenes and metrics, with a commanded hazard trajectory defined over the episode. Each moving configuration runs 624 episodes per condition, with 48 initial states across 13 scenes. The orbit uses a separate set of 13 scenes on which a 25 mm orbit is feasible; nine of them are shared with the other conditions. The six conditions are a stationary hazard, a circular orbit of 25 mm radius, a recurrent 300 mm shuttle, and one-way escape trajectories of 50, 150, and 300 mm that hold at their endpoints. All moving trajectories use a commanded speed of 50 mm/s. Each episode records the hazard’s true pose, commanded pose, and per-camera visibility at every step.

Reported mean rates average the six hazard-motion conditions. The repository baselines run all six conditions with the same episode keys (Sec. V). The ablations (Sec. VI) and the policy-configuration comparison (Sec. V-B) use the 300 mm escape condition, whose reset-to-final displacement is the largest, so a stale hazard estimate loses the most over the episode. Every simulated configuration uses the public $\pi_{0.5}$ LIBERO checkpoint (pi05-libero) [1]. It runs in its released serving configuration with three camera slots, 200 text tokens, and ten integration steps. Edge timing and energy

TABLE I

STATIC AND MOVING-HAZARD PERFORMANCE. C/S/SS: COLLISION/TASK SUCCESS/SAFE-SUCCESS (%); BOLD: BEST PER ROW IN (A); UNDERLINE: BEST PER BLOCK IN (B). EE: END-EFFECTOR; BOTH COVERAGE VARIANTS HOLD THE HAZARD ESTIMATE FROZEN AT RESET. SHIELD-INACTIVE EPISODES STAY IN EVERY DENOMINATOR. DISTANCES: MM; MEAN: OVER THE SIX CONDITIONS. [§]ADAPTED FROM THE AUTHORS’ CODE (SEC. IV).

(a) Moving benchmark: C↓ / S↑ / SS↑					(b) Static SafeLIBERO			
Condition	$\pi_{0.5}$ (unshielded)	AEGIS [§]	OSCBF [§]	Ours	Method	C↓	S↑	SS↑
Escape 300	45.99 / 67.63 / 42.15	29.17 / 62.66 / 46.63	31.73 / 69.87 / 53.53	22.44 / 64.58 / 56.41	$\pi_{0.5}$ (unshielded)	83.44	60.00	15.06
Escape 150	53.04 / 65.38 / 39.58	40.22 / 53.69 / 37.50	41.83 / 64.90 / 45.67	24.52 / 58.49 / 49.04	AEGIS [§]	30.75	<u>64.88</u>	49.75
Escape 50	72.76 / 56.41 / 23.88	42.47 / 58.17 / 39.42	57.69 / 58.65 / 33.49	25.32 / 55.29 / 44.07	OSCBF [§]	66.88	60.56	27.88
Shuttle 300	59.62 / 68.59 / 37.18	41.67 / 63.14 / 43.91	44.71 / 68.75 / 47.92	36.54 / 62.82 / 51.92	Ours	<u>27.00</u>	60.31	<u>51.69</u>
Orbit 25	78.85 / 66.03 / 18.59	38.14 / 74.68 / 50.32	59.94 / 68.43 / 36.06	33.65 / 66.83 / 48.72	Multi-link, frozen	25.19	61.62	<u>52.88</u>
Stationary	83.49 / 53.53 / 14.74	22.76 / 62.98 / 51.60	57.85 / 60.42 / 34.62	21.15 / 60.26 / 52.40	EE coverage	31.31	<u>65.62</u>	51.56
Mean	65.62 / 62.93 / 29.35	35.74 / 62.55 / 44.90	48.96 / 65.17 / 41.88	27.27 / 61.38 / 50.43				

use the reduced configuration of Sec. III-D, whose effect on outcomes Sec. V-B is measured with the shield active.

Metrics. Safety is measured through collision rate and reported alongside task success and safe-success, defined as task completion without collision. For comparability with prior work, the evaluation adopts the SafeLIBERO displacement-based collision criterion of the released AEGIS benchmark code [8], written *collision* throughout: an episode is flagged when the sum of absolute hazard position differences along the three axes exceeds 1 mm at any control step in MuJoCo. Displacement is measured from the initial hazard position in static scenes and from the latest commanded position in moving scenes, excluding scripted motion from the latter. The criterion records a disturbance of the hazard, which need not involve robot–hazard contact, so every episode also records MuJoCo robot–hazard contacts. Across the six hazard-motion conditions, 82.05% of the unshielded policy’s flagged episodes carry a recorded contact, against 48.29% under our shield: shielding removes contact episodes before it removes light disturbances. Rates on this stricter endpoint are therefore reported alongside the displacement endpoint (Sec. V-A).

For the static benchmark, a replay of the recorded actions of the unshielded, coverage-variant, and complete-system configurations recovers peak hazard displacement and recomputes collision at 10 mm to assess sensitivity to collision severity.

Implementation The simulated robot is the Franka Panda of the LIBERO suites in MuJoCo. The hazard namer is Qwen3-VL, and the detector is Grounding DINO [16], prompted with the benchmark’s original caption, an oracle-corrected caption, or the VLM-generated name used in deployment. A fixed scale factor converts the policy’s normalized delta-pose actions into the end-effector twists the program uses, and converts the filtered twist back before execution. The barrier program (2) is solved with OSQP through CVXPY, weighting the twist deviation by $W = I_6/25$ and each guarded ellipsoid’s auxiliary rate by $W_z = I_3$.

Baseline implementations AEGIS [8] and OSCBF [13] run from the authors’ public code, integrated with the same policy, simulator, and benchmark episodes. AEGIS retains its end-effector barrier and two-view geometry fitted at reset, with its GLM-4.5V hazard namer replaced by Qwen3-VL. OSCBF retains torque-level filtering and its robot-sphere

model, configured to match the simulated robot’s dynamics and torque limits, and takes hazard geometry and motion from our original caption perception pipeline. The robot envelopes therefore differ across methods. OSCBF keeps its published spheres and gains without tuning; its spheres end short of the open fingertips, whereas the AEGIS hand ellipsoid and ours enclose the gripper with margin.

Statistical comparisons. All configurations run the same episodes with the same policy sampling seed, so comparisons are paired. Episodes from a single scene are correlated, so per-episode differences are averaged within each scene (13 moving, 32 static) and these scene averages serve as the samples; a mean over conditions exchanges the two configurations of a scene jointly in every condition that contains it, across the 17 scenes the six conditions span. A difference is significant when a paired permutation test across the scene averages yields $p < 0.05$ after Holm–Bonferroni correction [27] for the differences claimed in the same sentence. Two configurations are equivalent when the 90% confidence interval of their difference lies within ± 3 percentage points, a margin set before the experiments.

Timing and energy. The control loop targets 20 Hz. After warmup, filter latency is measured per control step and policy latency per chunk invocation; the median and the 99th percentile (p50 and p99) are reported at these respective cadences. Policy-configuration comparisons run on the development platform, where call times are medians of within-episode percentiles; edge timing is measured on a Dell XPS 14 with an Intel Core Ultra X7 358H (Panther Lake), an iGPU, and an NPU. Energy and average power are read from the system-scope RAPL `psys` counter, a platform total with no per-device attribution, and are reported per isolated invocation above the 14.5 W idle power.

All reported rates are computed in simulation. The physical SO-101 evaluation in Sec. V-B reports episode counts instead, with an external Intel RealSense D455 feeding both the policy and the shield. Each episode is scored by observation: a success when the object reaches its target, and a contact when the arm touches the hazard.

V. EVALUATION

This section first compares safety and task performance across static and moving hazards, then evaluates edge execution and policy optimizations with the shield active.

TABLE II

EXECUTION ON THE EDGE PLATFORM. (A) IN-LOOP MEDIANS PER POLICY CALL AND ITS PREFIX ENCODING, CACHE, AND INTEGRATION LOOP ON THE CPU, iGPU, AND NPU. (B) SHIELD COMPONENTS. POWER: MEAN ABOVE THE IDLE FLOOR OVER AN ISOLATED INVOCATION; ENERGY: ABOVE IDLE, PER ISOLATED INVOCATION.

(a) Policy call by device						
Device	Call (ms)	Encode (ms)	Cache (ms)	Loop (ms)	Power (W)	Energy (J)
CPU	7844.2	-	-	-	38.5	296
iGPU	177.3	33.2	78.6	65.2	44.0	7.86
NPU	446.2	135.0	242.5	68.7	23.8	8.84
(b) Shield components on the deployed mapping						
Component	Device	Cadence	p50 / p99 (ms)	Energy (J)		
Barrier (5 elements)	CPU	step	2.0 / 2.2	0.0189		
Optical flow	CPU	stride 5	3.1 / 3.6	0.132		
Detector (2 views)	iGPU	reset	457.8 / 549.4	19.7		
Hazard namer	iGPU	reset	1146.0 / 1153.8	51.1		

AEGIS and OSCBF are external reference points that keep their own robot envelopes and gains, so this comparison establishes practical standing.

A. Safety and Task Performance

Our shield reduces collisions and increases the rate of safe task completion around moving hazards. Collision falls from 65.62% to 27.27% and safe-success rises from 29.35% to 50.43% (Table Ia); task success, 62.93% unshielded and 61.38% with the shield, shows neither a significant change nor equivalence within the ± 3 percentage-point margin. Against AEGIS, our shield collides significantly less often, in 27.27% of episodes compared to 35.74%, while its safe-success rate of 50.43%, compared to 44.90%, is not significantly higher.

OSCBF also reduces collisions relative to the unshielded policy, but exceeds our shield’s collision rate by 8.2 to 36.7 percentage points in all six conditions. With the original-caption perception, OSCBF collides in 48.96% of episodes and reaches 41.88% safe-success, both significantly worse than ours; its task success of 65.17%, compared to 61.38%, shows neither a significant difference nor equivalence. The OSCBF gap does not show that torque-level filtering protects less than action-level filtering, because the robot envelopes differ. With the hazard enclosed in a bounding sphere instead of the fitted ellipsoid, OSCBF collides in 25.80% of 300 mm escape episodes, and none of its three rates differ significantly from those of the complete system.

Counting only episodes with a recorded robot-hazard contact lowers every rate and leaves the ranking of the four systems unchanged: across the six conditions, collision is 53.85% unshielded, 31.86% for OSCBF, 23.61% for AEGIS, and 13.17% for our shield.

On static SafeLIBERO, our shield collides in 27.00% of episodes, compared to 30.75% for AEGIS. The multi-link configurations have the highest observed safe-success (Table Ib). On this still hazard, optical-flow transport adds no measured benefit: the multi-link configuration with a frozen hazard estimate collides in 25.19% of episodes,

compared to 27.00% for the complete system. OSCBF collides in 66.88% of static episodes (Table Ib).

B. Edge Deployment

The barrier and the optical-flow tracker run on the CPU at their respective step cadences; the policy, the detector, and the hazard namer run on the iGPU, with the latter two running only at reset (Table II). A barrier step costs 0.0189 J, and an optical-flow update costs 0.132 J, compared to 7.86 J for a policy call. The detector and the hazard namer add 19.7 J and 51.1 J, respectively, once per episode.

In its released serving configuration, the policy takes 343 ms per call on the iGPU. Trimming the prefix from 968 to 576 positions and halving the integration steps gives the deployed 177.3 ms, with the policy weights unchanged.

On the NPU, the prefix dominates the call: prefix cache 54.3% and prefix encoding 30.3%, compared to 15.4% for the integration loop. The unbatched policy’s matrix shapes suit the NPU poorly, so its call is $2.52\times$ that of the iGPU. The iGPU draws 44.0 W against 23.8 W on the NPU but completes each call sooner, so it spends 7.86 J per call against 8.84 J. The detector’s fusion encoder is slower still on the NPU, at 1,837 ms against 105 ms on the iGPU.

Amortized over a ten-action chunk, with a barrier step at every control step and an optical-flow update at every fifth, the deployed mapping demands 20.3 ms per 50 ms control period, but each chunk boundary stalls for the duration of the policy call, 177.3 ms at p50, when calls are issued synchronously.

Safety under policy optimization For static hazards, the reduced configuration of Sec. III-D leaves task success and safe-success equivalent. At the same time, collision rises by 1.75 percentage points. With the filter active, the same reduction lowers the single-client call time by 32.7% at p50. On the 300 mm escape condition, the same reduction leaves shielded collision at 22.92% against 22.44%, so the shield’s collision reduction is unaffected by the policy configuration. Task success falls by 4.17 percentage points and safe-success by 3.53 percentage points, both significant. Without the shield, the same reduction changes neither rate significantly. The barrier corrects about a quarter of control steps in every configuration. That fraction is insensitive to the deviation threshold used to define a correction, so barrier activity is governed by geometry rather than by the policy configuration.

Physical deployment We also deploy the shield on an SO-101 arm across four tabletop placement tasks, with instructions such as *place the white sugar cube in the cup*, with a bottle or a box obstructing the arm’s path. A fixed $\pi_{0.5}$ policy provides the nominal actions. An external Intel RealSense D455 supplies aligned RGB-D observations for fitting the hazard ellipsoid at reset and the image stream on which optical flow transports it; its RGB stream and a wrist-mounted camera provide the policy observations. On the same edge platform and device mapping as in Table II, the transported ellipsoid follows the hazard as it is carried across the arm’s path (Fig. 3). Across the four tasks, four episodes each, the

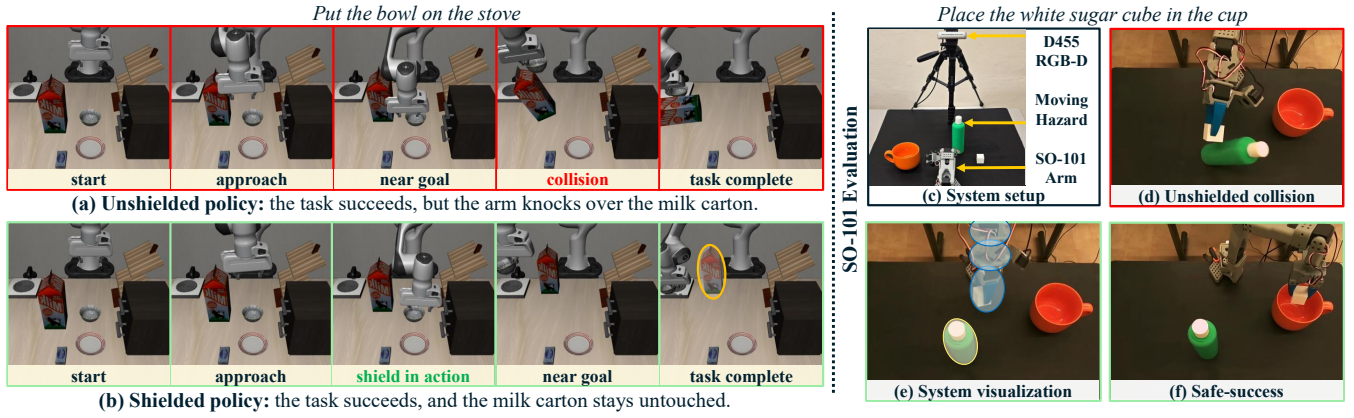


Fig. 3. **The shield in simulation and on hardware.** (a,b) MuJoCo, 300 mm escape condition: both executions complete the task, but only the shielded execution leaves the moving hazard untouched, a safe-success. (c–f) One of the four tasks on a physical SO-101 arm driven by the same edge platform.

TABLE III

WHERE PERCEPTION AND TRACKING LOSE PROTECTION. 300 MM ESCAPE CONDITION. BOLD: BEST RATE PER COLUMN; †: DETECTOR PROMPTED WITH THE ORACLE-CORRECTED CAPTION.

Configuration	Coll. (%)↓	Succ. (%)↑	Safe-succ. (%)↑
Unshielded $\pi_{0.5}$	45.99	67.63	42.15
Frozen hazard estimate	30.29	58.81	45.51
Fixed-cadence refresh	30.13	58.97	45.83
Exact geometry + true motion†	18.43	65.87	60.10
Detector + true motion†	20.19	68.59	60.90
Detector + flow†	22.44	64.74	56.57
Ours (VLM-generated name)	22.44	64.58	56.41

†Not deployable. The frozen-estimate and fixed-cadence variants use the original caption, so comparisons between the upper and lower blocks are system-level rather than tracking-only.

arm touched the hazard in 3 of 16 shielded episodes and in 11 of 16 unshielded ones; it completed the placement in 11 shielded episodes and in 13 unshielded ones. The hardware episode budget is small, and the hazard is hand-carried, so these counts are reported without the paired tests of Sec. V-A.

VI. ABLATIONS AND ANALYSIS

This section examines the protection and task costs of multi-link coverage, losses due to perception and tracking, and the placement accuracy and update frequency required to maintain protection. Each study varies one policy factor, with the perception and controller otherwise fixed, so these are the paper’s controlled comparisons along the coverage and motion axes.

A. Multi-Link Protection and Its Task Cost

With perception and the controller held fixed, guarding the wrist and forearm in addition to the end effector reduces static collision from 31.31% to 25.19% (Table Ib). Neither task success (65.62% to 61.62%) nor safe-success (51.56% to 52.88%) changes significantly. Relative to unshielded execution, safe failures (episodes that avoid the hazard but do not complete the task) rise from 1.50% to 21.94% of episodes under multi-link coverage. With exact hazard geometry and motion, which is not deployable, the same coverage change reduces collision from 21.94% under end-effector coverage to 14.44%.

Shielding also reduces collision severity: unshielded episodes displace the hazard by 147.7 mm at the median, and more than nine in ten exceed 10 mm, whereas the disturbances surviving multi-link coverage fall to 32.5 mm. When a collision requires 10 mm of displacement instead of 1 mm, multi-link coverage retains 83.7% of its advantage over end-effector coverage (5.12 versus 6.12 percentage points), a benefit concentrated at displacements below 50 mm.

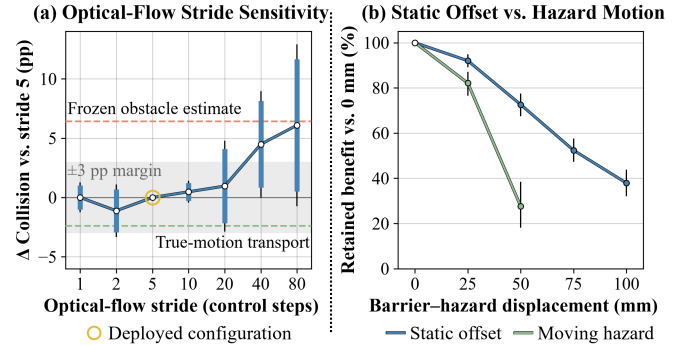


Fig. 4. (a) Per-step optical flow is equivalent to stride 5 within the shaded ± 3 pp margin (pp: percentage points). (b) Motion increases placement sensitivity: the hazard leaves an ellipsoid frozen at reset; both displacement sweeps use the static benchmark. Thin bars: 95% CIs; thick: 90%.

B. Hazard Estimation and Tracking Failures

Broader coverage is worth only as much as the placement of the hazard ellipsoid it works against. Exact hazard geometry and motion are therefore replaced by staged perception to locate where protection is lost. On the 300 mm escape condition, collision falls from 30.29% with a hazard estimate frozen at reset to 18.43% with exact geometry and true motion, which is not deployable (Table III). Replacing the exact geometry with the detector geometry while retaining true motion raises the collision rate to 20.19%. At the same time, task success and safe-success are both higher, so exact geometry and motion do not upper-bound task outcomes. Replacing true motion with optical flow raises collision to 22.44% and reduces task success from 68.59% to 64.74%. Optical flow accounts for the remaining measured loss and still improves every rate over the frozen-hazard estimate. Image-based perception therefore retains most of the protection available with exact geometry and motion.

Replacing the oracle-corrected caption with the VLM-generated name leaves the aggregate collision unchanged at 22.44%, with negligible changes in task success and safe-success. This parity conceals a failure on the object with the weakest grounding. In six of 624 episodes, the same plausible but incorrect phrase causes the filter guard to assign a different object, a systematic failure that aggregate rates hide.

Repeated detection incurs both computation and association costs. One detector call on the edge platform costs about $148\times$ the latency and $149\times$ the energy of an optical-flow update (Table II). Refits are accepted only within 50 mm of the last accepted position; this gate rejects correct proposals after larger motion and can retain a wrong anchor. The fixed-cadence variant therefore collides as often as a hazard estimate frozen at reset (Table III), accepting at least one refresh in only 184 of 624 episodes. On the stationary condition, repeated refitting raises collision by 8.81 percentage points over the frozen estimate: the frame-to-frame shape variation that a single reset fit avoids. Optical flow instead transports the identity established at reset, updating the ellipsoid without re-detection or association.

C. Placement Accuracy and Update Frequency

Two sweeps test the placement accuracy and update frequency needed to maintain protection. Retained benefit is collision reduction relative to unshielded execution, normalized by the reduction at zero displacement. Both sweeps run on the static benchmark (Fig. 4(b)). Translating only the ellipsoid retains 92.0% and 72.6% of its benefit at 25 and 50 mm; moving the hazard away from a frozen estimate retains 82.1% and 27.6%. The gap between the two widens from 9.9 to 45.0 percentage points across that interval, so a good reset fit does not replace transport during motion. The median fitted-center error on the static benchmark, 20.8 mm, is below the smallest nonzero offset tested.

The cadence sweep uses 624 paired 300 mm escape episodes per configuration with the original caption, separate from the oracle-corrected-caption variants above (Fig. 4(a)). At strides 1, 2, 5, 10, and 20, collision stays between 22.76% and 24.84%, strides 1 and 2 are equivalent to stride 5, and neither task success nor safe-success differs significantly from stride 5. True-motion transport stays lower at 21.47%, and more frequent updates do not close this gap, because optical flow localizes the hazard with 48.2 to 49.7 mm of error at every stride, compared to 37.3 mm with true motion. Longer strides lose protection: collision rises to 28.37% at stride 40 and 29.97% at stride 80, close to the frozen hazard estimate at 30.29%.

On the matched stationary condition, no stride reduces collision below the frozen estimate, and per-step measurement is equivalent to stride 5: more frequent measurement cannot help a static hazard. Stride 5 is therefore deployed, at 3.1 ms per measurement on the edge CPU.

VII. CONCLUSION AND FUTURE DIRECTIONS

This work shows that runtime protection for pretrained VLA policies can be extended beyond the end effector, kept

current around moving hazards, and executed on shared edge hardware without retraining the policy. Multi-link coverage improves protection across the arm, while sparse optical flow preserves most of the benefit lost when the hazard estimate is frozen. Both the barrier and tracker remain millisecond-scale workloads, making policy inference the dominant cost of the deployed system rather than shielding.

The edge measurements also identify where that cost lies. Prefix trimming and fewer flow-matching integration steps substantially reduce policy latency, while NPU execution is still dominated by vision-language prefix encoding and cache construction. For this unbatched $\pi_{0.5}$ workload, lower accelerator power does not translate to lower energy per call; reducing prefix-processing latency is therefore the more relevant optimization target.

A remaining challenge is tighter interaction between the policy and shield. The shield modifies commanded motion, but the fixed policy does not observe those corrections within its current action chunk, which can limit recovery when avoidance requires replanning. Feeding shielded state and corrective actions back into the policy, together with safe fallback behaviors, offers a path toward higher task completion and stronger protection under moving hazards. More broadly, the results demonstrate that motion-aware, multi-link shielding can provide a practical onboard safety layer around general-purpose manipulation policies.

REFERENCES

- [1] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proc. Conf. Robot Learn. (CoRL)*, 2025, pp. 17–40.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proc. Conf. Robot Learn. (CoRL)*, 2024, pp. 2679–2713.
- [4] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 2165–2183.
- [5] R. Cui, Z. Zhang, J. Pang, H. Chi, J. Guo, S. Zhang, S. Xie, X. Jin, Y. Mu, J. Yang, G. Yao, X. Zhan, Y.-Q. Zhang, and H. Zhao, “LIBERO-Safety: A comprehensive benchmark for physical and semantic safety in vision-language-action models,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2026, pp. 512–530.
- [6] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [7] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *Proc. Eur. Control Conf. (ECC)*, 2019, pp. 3420–3431.
- [8] S. Hu, Z. Liu, S. Liu, J. Cen, Z. Meng, S. Wang, X. Li, and X. He, “VLSA: Vision-language-action models with plug-and-play safety constraint layer,” 2025, arXiv:2512.11891.
- [9] S. Park, F. Zhang, B. Mirzasoleiman, S. Talebi, and N. Sehatbakhsh, “Your model already knows: Attention-guided safety filter for vision-language-action models,” 2026, arXiv:2606.09749.
- [10] S. Dean, A. J. Taylor, R. K. Cosner, B. Recht, and A. D. Ames, “Guaranteeing safety of learned perception modules via measurement-robust control barrier functions,” in *Proc. Conf. Robot Learn. (CoRL)*, 2020, pp. 654–670.

- [11] W. English, H. Zheng, and R. Ewetz, "Neuro-symbolic safety guidance for vision-language-action models via constrained flow matching," 2026, arXiv:2607.01378.
- [12] K. Sinaei, H.-C. Wu, and D. Ebeigbe, "Safe vision language action models via barrier enhanced flow matching," 2026, arXiv:2607.29569.
- [13] D. Morton and M. Pavone, "Safe, task-consistent manipulation with operational space control barrier functions," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 187–194.
- [14] Y. Xiong, D.-H. Zhai, and Y. Xia, "Robust whole-body safety-critical control for sampled-data robotic manipulators via control barrier functions," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 16 050–16 061, 2025.
- [15] A. Beaudin, H. Krasowski, K. Nagpal, S. A. Seshia, M. Arcak, and N. Mehr, "Any-Body Guard: Universal safeguarding for manipulation policies via action masking," 2026, arXiv:2606.22278.
- [16] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 38–55.
- [17] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. R  dle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Doll  r, and C. Feichtenhofer, "SAM 2: Segment anything in images and videos," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [18] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 1981, pp. 674–679.
- [19] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," 2025, arXiv:2506.01844.
- [20] P. Budzianowski, W. Maa, M. Freed, J. Mo, W. Hsiao, A. Xie, T. Mioduchowski, V. Tipnis, and B. Bolte, "EdgeVLA: Efficient vision-language-action models," 2025, arXiv:2507.14049.
- [21] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [22] H. Li, W. Mao, Z. Lan, H. Xiong, H. Wang, C. Si, Z. Liu, X. Deng, and H. Chen, "BFA++: Hierarchical best-feature-aware token prune for multi-view vision language action model," *IEEE Robot. Autom. Lett.*, vol. 11, no. 5, pp. 6002–6009, May 2026.
- [23] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, Y. Lei, and M. Zhang, "QuantVLA: Scale-calibrated post-training quantization for vision-language-action models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [24] K. Zhou, Q. Chen, D. Peng, Z. Li, X. Li, and J. Gu, "Characterizing vision-language-action models across XPU: Constraints and acceleration for on-robot deployment," 2026, arXiv:2604.24447.
- [25] R. Funada, K. Nishimoto, T. Ibuki, and M. Sampei, "Collision avoidance for ellipsoidal rigid bodies with control barrier functions designed from rotating supporting hyperplanes," *IEEE Trans. Control Syst. Technol.*, vol. 33, no. 1, pp. 148–164, Jan. 2025.
- [26] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023, pp. 44 776–44 791.
- [27] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.